



Hacettepe Üniversitesi İstatistik Bölümü İST281 Bilgisayar Programlama Dersi

Demetler, kayıtlar (Tuples)

- Demet değiştirilemeyen (immutable) liste yapısıdır. Oluşturulduktan sonra ekleme, silme, değiştirme ve sıralama gibi işlemler yapılamaz. Sadece okunarak kullanılabilir. Küçük parantezler () ile gösterilir.
- Demet yapısı, liste yapısına göre bellekte daha az yer tutar.
- Demet ve liste yapısının kullanımı birbirine çok benzer. Ancak liste yapısında değişiklik yapan işlemler demet yapısında kullanılamaz.
- Demet yapısı, liste ve sözlük gibi diğer yapıların içinde kullanılabilir.

- **Boş demet oluşturmak:**

t=tuple()

veya

t=()

- **Değer aktararak demet oluşturmak:**

t=(1,2,3)

print(t,type(t))

(1, 2, 3) <class 'tuple'>

t=([1,2,3,4])

print(t,type(t))

[1, 2, 3, 4] <class 'list'>

t=("sıra no",1,2.5,1)

print(t)

('sıra no', 1, 2.5, 1)

- **Erişim:**

```
t=(1,2,3)
```

```
a=t[1]
```

```
print(a)
```

2

```
t=(1,2,3)
```

```
a=t(1)
```

```
print(a)
```

TypeError: 'tuple' object is not callable

```
t=(1,2,3,4)
```

```
a=t[1:3]
```

```
print(a)
```

(2, 3)

- **Değişiklik yapılamaz:**

```
t=(1,2,3)  
t[0]=5
```

TypeError: 'tuple' object does not support item assignment

- **Demet üzerinde değişiklik yapmayan fonksiyon ve metotlar liste yapısında olduğu gibi kullanılabilir.**

Örneğin: len(), min(), max(), sum() fonksiyonları ve .count(), index() metotları kullanılabilir.

- **Demetten listeye dönüşüm:**

```
t=(4,7,8,"a")  
l=list(t)  
print(l, type(l))
```

```
[4, 7, 8, 'a'] <class 'list'>
```

- **Listeden demete dönüşüm:**

```
l=[4,7,5,8]  
t=tuple(l)  
print(t,type(t))
```

```
(4, 7, 5, 8) <class 'tuple'>
```

- **Metinden demete dönüşüm:**

```
s="Python"  
t=tuple(s)  
print(t, type(t))
```

```
('P', 'y', 't', 'h', 'o', 'n') <class 'tuple'>
```

- **Parantez gerekli değildir:**

```
t=1,2,3,"y"
```

```
print(t, type(t))
```

```
(1, 2, 3, 'y') <class 'tuple'>
```

Sözlükler (Dictionaries, dict)

- Anahtar (key) ve Değer (value) ikilileriyle (mapping) oluşturulan yapılardır.
- Sözlük yapısı, değiştirilebilir (mutable) özelliktedir.
- Büyük parantez {} simgesi ile gösterilir.
- Anahtar değerleri tekrar edemez (unique).
- Anahtar değerleri, sayı, metin veya demet olabilir. Liste olamaz.
- Sözlük içinde, bağımsız alt sözlükler olmaz.

- **Boş sözlük oluşturmak:**

```
sözlük={}
```

```
print(type(sözlük))
```

```
<class 'dict'>
```

```
sözlük=dict()
```

```
print(type(sözlük))
```

```
<class 'dict'>
```

- **Değer aktararak sözlük oluşturmak:**

```
#sözlük={<key:value>, ....}
```

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
print(kalori, type(kalori))
```

```
{'pilav': 258, 'kumpir': 411, 'börek': 284, 'makarna': 220} <class 'dict'>
```

```
kalori=dict(pilav=258, kumpir=411, börek=284, makarna=220)
print(kalori, type(kalori))
```

```
{'pilav': 258, 'kumpir': 411, 'börek': 284, 'makarna': 220} <class 'dict'>
```

```
kalori=dict([("pilav",258),("kumpir",411),("börek",284),("makarna",220)])
print(kalori, type(kalori))
```

```
{'pilav': 258, 'kumpir': 411, 'börek': 284, 'makarna': 220} <class 'dict'>
```

```
x={x:x**2 for x in range(5)}
print(x, type(x))
```

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16} <class 'dict'>
```

- **Sözlük değerleri liste veya demet yapısında olabilir.**

```
s={1:(1,2,3), 2:(3,4,5), 3:(6,7,8)}  
print(s)
```

```
{1: (1, 2, 3), 2: (3, 4, 5), 3: (6, 7, 8)}
```

```
s={1:[1,2,3], 2:[3,4,5], 3:[6,7,8]}  
print(s)
```

```
{1: [1, 2, 3], 2: [3, 4, 5], 3: [6, 7, 8]}
```

- **Erişim:** Sözlükteki bir değere (value) erişim için anahtar kullanılır. İndis kullanılmaz.

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
x=kalori["kumpir"]
```

```
print(x)
```

411

Anahtar yok ise KeyError hatası ortaya çıkar:

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
x=kalori["pide"]
```

KeyError: 'pide'

- **.get(key[,mesaj]) metodu:** Sözlükte anahtar arar. Bulunursa değeri verir. Bulanamazsa belirtilen mesajı verir.

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
x=kalori.get("börek")  
print(x)
```

```
y=kalori.get("makarna","Sözlükte yok")  
print(y)
```

```
z=kalori.get("pide", "Sözlükte yok")  
print(z)
```

```
z=kalori.get("pide")  
print(z)
```

```
284  
220  
Sözlükte yok  
None
```

- **.items() metodu:** Sözlükte bulunan anahtar:değer çiftlerini verir.

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
print(kalori.items())
```

```
dict_items([('pilav', 258), ('kumpir', 411), ('börek', 284), ('makarna', 220)])
```

- **Döngü ile kullanılabilir:**

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
for anahtar, öge in kalori.items():
```

```
    print(anahtar,öge)
```

```
pilav 258  
kumpir 411  
börek 284  
makarna 220
```

- **Diğer listeleme seçeneği:**

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
for anahtar in kalori:
```

```
    print(anahtar, kalori[anahtar])
```

```
pilav 258  
kumpir 411  
börek 284  
makarna 220
```

- **Sözlüğe öge eklemek:**

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
kalori["pide"]=350
```

```
print(kalori)
```

```
{'pilav': 258, 'kumpir': 411, 'börek': 284, 'makarna': 220, 'pide': 350}
```

- **Bir anahtara ilişkin değeri değiştirmek:**

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
kalori["makarna"]=225
```

```
print(kalori)
```

```
{'pilav': 258, 'kumpir': 411, 'börek': 284, 'makarna': 225}
```


- **Sözlükten öge silmek:**

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
del kalori["börek"]
```

```
print(kalori)
```

```
{'pilav': 258, 'kumpir': 411, 'makarna': 220}
```

- **Sözlüğü başka bir sözlük ile güncellemek:**

```
fiyat={"pilav":150, "kumpir":250, "börek":200, "makarna":100}
```

```
yeni_fiyat={"pilav":170, "kumpir":290, "börek":230, "makarna":120}
```

```
fiyat.update(yeni_fiyat)
```

```
print(fiyat)
```

```
{'pilav': 170, 'kumpir': 290, 'börek': 230, 'makarna': 120}
```

- **Sözlüğü başka bir sözlük ile güncellemek:** (anahtar yok ise)

```
fiyat={"pilav":150, "kumpir":250, "börek":200, "makarna":100}  
yeni_fiyat={"pilav":170, "kumpir":290, "börek":230, "pide":120}  
fiyat.update(yeni_fiyat)  
print(fiyat)
```

```
{'pilav': 170, 'kumpir': 290, 'börek': 230, 'makarna': 100, 'pide': 120}
```

- **Sözlük anahtarlarını ve değerlerini listelemek:**

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}  
k=kalori.keys()  
print(k, type(k))  
v=kalori.values()  
print(v, type(v))
```

```
dict_keys(['pilav', 'kumpir', 'börek', 'makarna']) <class 'dict_keys'>  
dict_values([258, 411, 284, 220]) <class 'dict_values'>
```

- **Sözlükte anahtar aramak:**

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
varmi= "börek" in kalori
```

```
print(varmi)
```

```
varmi= "pide" in kalori
```

```
print(varmi)
```

True
False

- **Sözlük öğelerinin hepsini silmek:**

```
kalori={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
kalori.clear()
```

```
print(kalori)
```

{}

- **Sözlüklerin eşitliği:**

```
kalori1={"pilav":258, "kumpir":411, "börek":284, "makarna":220}
```

```
kalori2={"kumpir":411, "börek":284, "pilav":258, "makarna":220}
```

```
aynımı=kalori1 is kalori2
```

```
print(aynımı)
```

```
aynımı=kalori1 == kalori2
```

```
print(aynımı)
```

False

True

Küme (Set)

- Sırası önemli olmayan ve tekrar etmeyen öğelerden oluşur.
- Basit kullanımı, üyelik testi ve tekrar eden öğelerin kaldırılmasıdır.
- Birleşim (union), kesişim (intersection), fark (difference) ve simetrik fark (symetric difference) gibi matematiksel işlemleri sağlar.
- Simgesi büyük parantezlerdir {}.
- Erişim için indis kullanılmaz.

- **Boş küme oluşturmak:**

```
küme=set()
```

```
print(type(küme))
```

```
<class 'set'>
```

- **Değer aktararak küme oluşturmak:**

```
küme={1,2,3,4}
```

```
print(küme, type(küme))
```

```
{1, 2, 3, 4} <class 'set'>
```

```
sepet={"portakal","elma","portakal","armut","elma","muz"}
```

```
küme=set(sepet)
```

```
print(küme)
```

```
{'muz', 'armut', 'portakal', 'elma'}
```

- **Küme işlemleri:**

- **Kümeye öge ekleme:**

```
sepet={"portakal","elma","armut"}  
sepet.add("muz")  
print(sepet)
```

```
{'elma', 'muz', 'portakal', 'armut'}
```

- **Kümeden öge çıkarma:** (.remove() metodu ile)

```
sepet={"portakal","elma","armut"}  
sepet.remove("elma")  
print(sepet)
```

```
{'armut', 'portakal'}
```

```
sepet={"portakal","elma","armut"}  
sepet.remove("muz")  
print(sepet)
```

```
KeyError: 'muz'
```

- **Kümeden öge çıkarma:** (.discard()) metodu ile

```
sepet={"portakal","elma","armut"}  
sepet.discard("elma")  
print(sepet)
```

```
{'armut', 'portakal'}
```

```
sepet={"portakal","elma","armut"}  
sepet.discard("muz")  
print(sepet)
```

```
{'elma', 'portakal', 'armut'}
```


- **Küme işlemleri:**

- **Tekrarlayan öğeler bulunmaz:**

```
a = set('abracadabra')
```

```
print(a)
```

```
b = set('alacazam')
```

```
print(b)
```

```
{'d', 'r', 'a', 'b', 'c'}  
{'l', 'a', 'z', 'm', 'c'}
```

- **Birleşim (union):** a veya b kümesinde veya her ikisinde de bulunan öğeler

```
a = set('abc')
```

```
b = set('abd')
```

```
c=a.union(b)
```

```
print(c)
```

```
# veya
```

```
c=a | b
```

```
print(c)
```

```
{'a', 'b', 'd', 'c'}
```

- **Kesişim (intersection):** a ve b kümesinin her ikisinde de bulunan öğeler

```
a = set('abc')
```

```
b = set('abd')
```

```
c=a.intersection(b)
```

```
print(c)
```

```
# veya
```

```
c=a & b
```

```
print(c)
```

```
{'b', 'a'}
```

- **Küme farkı (difference):** a kümesinde bulunan fakat b kümesinde bulunmayan öğeler

```
a = set('abc')  
b = set('abd')  
c=a.difference(b)  
print(c)  
# veya  
c=a - b  
print(c)
```

{'c'}

- **Simetrik küme farkı (symmetric difference):** a veya b kümesinde bulunan fakat her iki kümede birlikte bulunmayan öğeler

```
a = set('abc')  
b = set('abd')  
c=a.symmetric_difference(b)  
print(c)  
# veya  
c=a ^ b  
print(c)
```

```
{'d', 'c'}
```

Sorular

- **Soru 1:** Kullanıcı, klavyeden n adet sayı giriyor. Bu sayıları bir demet (tuple) yapısına aktarınız. Daha sonra girilen bir sayının demet içinde bulunup bulunmadığını arayan, varsa indis numarasını veren bir program yazınız.
- **Soru 2:** Yazacağınız programın başında $m \times n$ boyutlu bir örnek demet oluşturunuz. Matrisin her satırı büyükten küçüğe sıralı olacak biçimde yeni bir demet üretiniz, yeni demeti ekranda gösteriniz.

- **Soru 3:** Ekrandan veri girerek bir sözlük yapısı oluşturunuz. Kullanıcıdan önce sözlüğün öge sayısını isteyiniz. Sonra her öge için anahtar ve değer bilgilerini isteyiniz. Aldığınız bilgileri sözlüğe ekleyiniz.
- **Soru 4:** Program içinde değişik öge sayılarına sahip 3 tane sözlük oluşturunuz. Çeşitli anahtar ve değer bilgileri içeren 3 tane sözlüğü birleştirerek yeni bir sözlük oluşturunuz.
- **Soru 5:** Bir sözlük yapısında bulunan anahtar-değer çiftlerinden bazı *değerler* tekrar ediyor olsun. Sözlükte bulunan değerleri tekrar etmeyecek biçimde listeleyen bir program yazınız.
Örnek Sözlük: {"a":3,"b":5,"aa":3,"bb":5,"c":6,"ba":5}
Çıktı: {3,5,6}

- **Soru 6:** İki tane sözlükte, anahtar-değer çiftleri aynı olan öğeleri listeleyen bir program yazınız. Örnek sözlükleri kod içinde oluşturunuz.

Örnek:

sözlük1={1:"a",2:"b",3:"c",5:"e"}

sözlük2={1:"a",2:"c",3:"c",4:"d",5:"f"}

Çıktı:

1, 'a'

3, 'c'

- **Soru 7:** İki tane demet (tuple) veriliyor. 2. demette aynı olan öğeleri 1. demetten çıkaran bir program yazınız.

Örnek:

d1=(1,2,3,4,5)

d2=(4,5,6,7)

Çıktı:

1, 2, 3

Soru 8: Bir tane metin tipinde parametresi olan bir fonksiyon yazınız. Fonksiyon, metinden bir sözlük oluştursun ve geri döndürsün. Sözlüğün anahtarları harfler, değerleri ise o harfin metinde kaç kez tekrarlandığı olsun.

Örneğin: metin "ortalama" ise sözlük:

{"o":1, "r":1, "t":1, "l":1, "a":3, "m":1} olsun.